

REPOSITORY REVIEW

Asymptotic

A non-duplicative Wolfram 15 and Mathics3 audit

Three reproduced API defects,
narrowly scoped candidate fixes,
and a cross-runtime evidence assessment

Repository	VladimirReshetnikov/Asymptotic
Reviewed commit	8cee870994f5
Review date	September 10, 2026
Executed symbolic runtime	Wolfram Language 15.0.0, Linux x86-64
Mathics3 status	Source assessment; supplied tests not executed

Principal conclusion. Three independently reproduced defects remain at public API boundaries: named numeric constants are accepted as variables, Fourier residual options are consumed as a positional cutoff, and a proved conditional observable misses an available exponential-carrier implementation. Focused changes correct the demonstrated cases on Wolfram 15. No new critical coefficient or remainder theorem failure is asserted.

The review distinguishes executed results, source deductions, candidate fixes, and unexecuted acceptance tests. It does not reproduce the existing review backlog or claim a complete Mathics3 certification.

Contents

1	Executive assessment	3
2	Revision, evidence, and non-duplication	3
2.1	A fixed object of review	3
2.2	What was excluded	4
2.3	Evidence vocabulary	5
3	Implementation boundaries examined	5
3.1	Power-log data and exact carriers	5
3.2	Inverse families are not interchangeable	5
3.3	Realness, native information, and compatibility adapters	6
4	N1: numeric constants are accepted as variables	6
4.1	A public witness with a native comparison	6
4.2	Why this is an admission defect	7
4.3	Source cause	7
4.4	Candidate guard and its intended semantics	7
4.5	Executed candidate checks and remaining coverage	8
5	N2: Fourier residual options become a cutoff	8
5.1	A valid series, two superficially equivalent calls	8
5.2	How the signature creates the ambiguity	9
5.3	A narrow implementation change	9
5.4	Native candidate results	10
6	N3: a true condition loses the exponential-carrier route	10
6.1	The discrepancy	10
6.2	Two implementations of exponentiation, selected at the wrong stage	11
6.3	The required semantic invariant	11
6.4	Candidate change after condition proof	11
6.5	Executed results and scope limits	12
7	Numerical and structural controls	12
7.1	The 42-case screen	12
7.2	What the screen establishes, and what it does not	13
7.3	Patch checks are separate from arithmetic controls	13
8	Wolfram 15 and Mathics3: conclusions justified here	13
8.1	Wolfram 15	13
8.2	Mathics3	14
8.3	A focused acceptance matrix	14
9	Fix integration and focused development priorities	14
9.1	A staged source patch rather than an opaque replacement package	15
9.2	Recommended order of work	15
9.3	Performance implications without invented speedups	15
9.4	Precise completion criteria	16
10	Using the archive	16
11	Conclusion	16
A	Source coverage and claim boundaries	17
B	Condensed returned evidence	18

C Proposed additional acceptance cases**18**

1 Executive assessment

The most useful new result of this review is not another inventory of already documented mathematical and engineering limitations. It is a set of three concrete, reproducible boundary failures that survived the repository’s existing review process. Each has a short public-API witness, an identifiable source-level cause, a narrowly scoped implementation change, and an explicit distinction from the nearest previously registered topic.

ID	Finding	Observable consequence	Evidence
N1	Numeric constants accepted as source or target variables	A result object is produced for a request that cannot describe a variable approaching an endpoint	Reproduced on Wolfram 15; source and ordinary-inverse guards tested
N2	Fourier residual options captured as the optional cutoff	A normal options-only call returns <code>InvalidCutoff</code> ; adding <code>Automatic</code> makes it work	Reproduced; typed-signature change tested with ordinary, nested, and delayed options
N3	A proved condition bypasses specialized observable dispatch	An exact exponential observable succeeds without a condition and fails with a true condition	Reproduced; post-proof dispatch change tested; false condition still refused

All three are assessed as **medium-priority correctness or usability defects**. N1 concerns malformed-input admission, not a demonstrated incorrect expansion for a legitimate independent variable. N2 and N3 reject valid requests that closely related calls already support. None of the three justifies claiming that the ordinary coefficient engine is generally unreliable.

The package has substantial explicit machinery for branch selection, real coefficient checks, precision propagation, exact-core inversion, factored carriers, finite Fourier coefficients, flat sectors, and quantitative forward bounds. This review inspected the relevant implementation boundaries rather than assuming that every specialized result is an ordinary Taylor series.[7, 8, 9, 10, 11, 14]

A separate 42-case Wolfram 15 diagnostic screen returned 41 nonexact expansions and one exact expansion, with no flags under the stated numerical ratio test. That is useful negative evidence against broad arithmetic regressions in the screened expressions. It is not an interval proof of their error estimates, nor a substitute for the repository’s full test suites.

The accompanying source-patch generator stages four exact text changes in three canonical files. It does not modify the input checkout. The changes cover the reproduced examples; they are *candidate patches*, not an audited replacement distribution. In particular, the N1 guards do not yet establish a uniform admission policy for every specialized constructor or native-dispatch route.

2 Revision, evidence, and non-duplication

2.1 A fixed object of review

The repository was pinned to commit

8cee870994f506b501bae3ea6bd4a3a7edb895c1.

The readable source and the generated standalone package were examined at that revision, rather than mixing a moving `main` branch with an older article. The standalone file downloaded by the Wolfram evaluator had 692,381 bytes and loaded successfully. The package reported 38 names in its public context after that load. These are observations of the reviewed artifact, not a claim about subsequent revisions.

The repository's README and kernel module guide supplied the intended user model and source organization. The reviewed README identifies the Wolfram 15 target. The Mathics compatibility modules were inspected as an independent execution path, not treated as evidence that all Wolfram behavior is inherited unchanged.[1, 2, 18]

The connected Wolfram service reported:

```
{"15.0.0 for Linux x86 (64-bit) (May 6, 2026)",
  "Linux-x86-64"}
```

Each service call used a fresh kernel. A package load therefore appears in each executed test batch. Intermittent transport failures occurred; batches without a returned result are not counted as successful tests. A local Mathics runtime was not available for this review, and Mathics execution through the connected service was not established.

2.2 What was excluded

The principal exclusion sources were the repository's code-review status register, the wave-three and wave-four intake documents, the wave-five index, and retained or retired-report descriptions in those records. They contain the prior issue identifiers used in the novelty comparison.[3, 4, 5, 6]

In addition, full-text lexical screens completed for twelve retained articles: reports 21, 22, 24, 25, 36, 37, 38, 39, 42, 43, 44, and 45. The screened terms covered numeric-variable admission, the Fourier residual API, and conditional observable dispatch. The matches returned by those completed screens were inspected for relevance; incidental bibliography and unrelated source-variable references were not treated as duplicates. The article sizes and scope are recorded in `evidence/review_scope.json`.

A broader all-article scan was attempted but did not return a completed result. Consequently, this report does **not** claim that every retained article was read in full, or that lexical absence alone proves novelty. The new findings are based on their distinct public witnesses and source causes after comparison with the registers and the completed screens. The supplied offline scanner makes further comparison against a local checkout straightforward.

ID	Nearest earlier register topic	Why the present finding is different
N1	W3-02, variable/option identity	N1 concerns an actual numeric constant used as a coordinate, not an ordinary variable whose name resembles an option. The proposed guard must preserve valid names rather than blacklist their spelling.
N2	W3-02 and W4-06, argument interpretation	The failing function is specifically <code>FourierInverseResidual</code> ; its unrestricted optional cutoff absorbs its own documented option rule. The earlier records concern other option-identity or positional forms.
N3	W3-09, conditional input and backend selection	The failure is inside <code>SeriesObservable</code> 's package-level dispatch. Both successful and failing examples use the package's own carrier algebra; no native backend fallback is needed.

The report does not reissue the historical findings as new defects, and it does not convert existing roadmap entries into additional “opportunities.” Its development recommendations are the concrete admission, signature, and dispatch work needed to complete these three fixes.

2.3 Evidence vocabulary

An *executed observation* is an output returned by the stated runtime. A *source deduction* follows from the inspected definitions but may not establish that every public route reaches them. A *mathematical check* is an explicit derivation, independent of whether a kernel returns the expected form. A *proposed acceptance test* is a test supplied for future execution and is not included in any passing-test count.

This distinction matters particularly for Mathics3. A syntactically portable Wolfram Language test file is not a Mathics test result. Likewise, a patch that works when inserted into the generated package on Wolfram 15 is not automatically a successful rebuild and validation of both canonical distributions.

3 Implementation boundaries examined

3.1 Power–log data and exact carriers

The ordinary series representation can be read schematically as

$$F(x) = b(x) + p(x) \left(\sum_{\beta < h} w(x)^\beta P_\beta(\log w(x)) + O\left(w(x)^\rho (1 + |\log w(x)|)^d\right) \right), \quad w(x) \rightarrow 0^+. \quad (1)$$

The coordinate, exact offset, exact prefactor, coefficient rows, remainder pair, assumptions, and domain are separate pieces of data. The cutoff belongs to the internal coefficient scale, not necessarily to powers of the displayed variable. `seriesData`, `seriesFlat`, `seriesMake`, and the explicit arithmetic operations enforce or consume this separation.[8]

The distinction explains N3. For $s(x) = 1/x$, exponentiation can produce the exact prefactor $e^{1/x}$ with a constant internal jet. It need not represent $e^{1/x}$ as a power–log polynomial. The generic forward-exponential helper and the specialized series-exponential operation therefore have different applicable domains. Selecting between them is part of correctness, not merely an optimization.

The precision bookkeeping was read with that representation in mind. A numerical comparison must divide by the complete recorded remainder scale, including any exact prefactor, rather than by x^h solely because the user supplied a cutoff h . The control screen in Section 7 uses the object’s remainder-scale field for this reason.

3.2 Inverse families are not interchangeable

The source review covered the ordinary inverse constructor, Lambert cores, exact-core marker inversion, ordered Gamma/Barnes inverse corrections, finite Fourier coefficients, flat sectors, and special-function adapters. Their internal gradings differ: source or target powers, logarithmic orders, marker depth, and exponential-sector degree are not the same notion of truncation.[7, 12, 10, 13, 9, 11, 15]

This inspection prevented several misleading audit conclusions. A complete marker coefficient is not necessarily a monomial in the target; an exact zero exponential sector is not the same as a finite algebraic truncation; and a formal Fourier residual is not a numerical inverse certificate. No finding below relies on conflating these representations.

The Fourier residual witness is deliberately modest: a monomial leading core with one strictly higher-power oscillatory perturbation. It does not require any unsupported leading oscillation, incommensurate flat scale, or difficult parameter ordering. The failure occurs before the coefficient checker receives a meaningful cutoff.

3.3 Realness, native information, and compatibility adapters

The special-function real-domain and parameterized-function modules were examined together with the Mathics assumption, simplification, Taylor, and algebra adapters. The code contains separate realness checks and finite-order expansion paths; a native symbolic result is not, by itself, the entire package contract.[\[16, 17, 18, 19, 20\]](#)

The interval evaluator, source-domain translation, special numerical checker, and finite Dirichlet bounds were also inspected. This review does not introduce a new certificate or asymptotic-validity defect in those components. The absence of a new finding is a scoped review result, not a proof that all branches are correct. The exact files and ranges examined are summarized in [Appendix A](#).

4 N1: numeric constants are accepted as variables

4.1 A public witness with a native comparison

On the pinned package and Wolfram 15, the following request returns a generalized series:

```
r = AsymptoticAnalysis`AsymptoticExpansion[
  1/(1 + Pi), {Pi, 0, 3}, "Backend" -> "Package";
{Normal[r], r["Variable"], r["Remainder"]}
```

The returned fields are

```
{1 - Pi + Pi^2, Pi, PowerLogRemainder[Pi, 3, 0]}
```

Wolfram also emits `SeriesData::sdatv`, stating that `Pi` is not a valid variable. The public package call nevertheless returns the series object. In the same runtime, the direct native call

```
Series[1/(1 + Pi), {Pi, 0, 3}]
```

does not evaluate to a series. This comparison is an executed result, rather than an inference from documentation alone. Wolfram's `Series` documentation gives the variable-and-center interpretation of its specification.[\[24\]](#)

An inverse target shows the same class of omission:

```
r = AsymptoticAnalysis`AsymptoticInverse[
  x + x^2, {x, 0}, {Pi, 3}];
{Normal[r], r["Variable"], r["Remainder"]}
```

returns

```
{Pi - Pi^2, Pi, PowerLogRemainder[Pi, 3, 0]}
```

with invalid-variable messages during optional native-series construction.

4.2 Why this is an admission defect

The polynomial $1 - t + t^2$ is a correct truncation of $(1 + t)^{-1}$ at an independent variable $t = 0$. The problem is not that the geometric-series formula is wrong. The request names the numeric constant π itself as the coordinate and asks that constant to approach zero. A numeric constant is not an independent variable whose smallness can define a germ.

One could design a separate formal-symbol interface in which every printed name is treated as an inert generator. That is not what this API or its returned evaluated expressions do. Substituting `Pi` into the result produces ordinary numeric expressions involving π . The object neither introduces a fresh formal generator nor records a formal reinterpretation of the constant.

The inverse example is analogous. The valid expansion is $g(y) = y - y^2 + O(y^3)$ for an independent small target y . A target field equal to `Pi` does not provide such a target neighborhood. Users who pass the wrong symbol obtain an apparently successful object alongside messages, rather than an actionable admission error.

The severity is therefore **medium**: the defect can hide a malformed call and produce misleading metadata, but it does not establish a wrong expansion for valid variable inputs. It should not be presented as a failure of Lagrange inversion.

4.3 Source cause

The canonical endpoint normalizer is `localCoordinate` in [AsymptoticAnalysis.wl](#), lines 613–630. It validates the expansion point and direction, then constructs a local substitution and a displayed coordinate. It does not validate that its first argument is a nonnumeric symbol. The ordinary inverse constructor's line 1011 checks only that source and target are distinct. A `_Symbol` pattern is insufficient because `Pi` is a symbol.^[7]

These are two different admission sites. Adding a source-chart guard alone fixes the forward witness but not the inverse target. This was checked during patch development: an initial change covering the chart and other paired guards still allowed the ordinary inverse target because that constructor used a separate equality-only check. The final candidate explicitly changes that check.

This is an important implementation lesson from the actual experiment. A grep for one guard spelling is not evidence that every constructor has been covered. The delivered patch consequently states its scope instead of claiming a universal validation repair.

4.4 Candidate guard and its intended semantics

The candidate introduces a private predicate:

```
validSeriesVariableQ[v_] :=
  MatchQ[v, _Symbol] && ! NumericQ[v] &&
  ! MemberQ[{"System`Glaisher", "System`Khinchin"},
    Context[v] <> SymbolName[v]];
```

It is used before constructing a source chart and in the ordinary inverse source/target check. The first condition protects the subsequent symbol operations. `NumericQ` rejects ordinary numeric named constants. The qualified-name exclusions are a narrow portability precaution for constants that can remain inert on an alternative evaluator; they are not an exhaustive registry or a tested `Mathics` result.

The comparison uses complete contexts, not just printed names. A user-defined `AuditVars`Pi` remains a

valid indeterminate. A blanket prohibition on all `System`` symbols would be an overcorrection and could reject otherwise usable symbols. The policy must also be explicit about evaluated symbols with values: a symbol that has already evaluated to a number must not be silently reinterpreted as an independent variable.

The corrected ordinary inverse check is:

```
If[! validSeriesVariableQ[x] ||
    ! validSeriesVariableQ[y] || x == y,
    fail["InvalidVariables",
        "Source and target variables must be distinct nonnumeric symbols."]];
```

The source-chart failure uses `InvalidVariable` and a corresponding message. No coefficients, precision pairs, or branch formulas are changed.

4.5 Executed candidate checks and remaining coverage

In a fresh Wolfram 15 kernel, the two N1 anchors each matched once. The patched forward Pi request returned `Failure["InvalidVariable",...]` and the ordinary inverse target returned `Failure["InvalidVariables",...]`. Two controls remained correct:

```
AsymptoticExpansion[1/(1 + AuditVars`Pi),
    {AuditVars`Pi, 0, 3}, "Backend" -> "Package"] // Normal
(* 1 - AuditVars`Pi + AuditVars`Pi^2 *)

AsymptoticExpansion[Sin[x], {x, 0, 5},
    "Backend" -> "Package"] // Normal
(* x - x^3/6 *)
```

The unqualified function names in this explanatory listing assume the package has already been loaded; the runnable artifact uses qualified names.

The next development step is to place the same admission policy at the relevant public boundaries, including specialized target constructors and native dispatch, and verify that failure cannot trigger a fallback that restores the malformed request. The delivered minimal patch is not claimed to complete that work. Tests for other numeric constants, assigned variables, public aliases, each specialized target, and inert Mathics constants are proposed acceptance cases, not completed runtime results.

5 N2: Fourier residual options become a cutoff

5.1 A valid series, two superficially equivalent calls

Construct a finite Fourier inverse:

```
s = AsymptoticAnalysis`AsymptoticFourierInverse[
    x + x^2 Sin[Log[x]], {x, 0}, {y, 3}];
```

On Wolfram 15, this options-only call fails:

```
AsymptoticAnalysis`FourierInverseResidual[
    s, "MaxTerms" -> 1000]
(* Failure["InvalidCutoff", ...] *)
```

The message says that the Fourier residual cutoff must be positive and exact. The user has not supplied a cutoff at all. Inserting an explicit placeholder succeeds:

```
AsymptoticAnalysis`FourierInverseResidual[
  s, Automatic, "MaxTerms" -> 1000]
(* <|"ZeroBelowCutoff" -> True,
  "ResidualBlocks" -> {},
  "NormalizedResidual" -> 0,
  "RelativeCutoff" -> 2, ...|> *)
```

The successful call provides a direct control: the series, budget, and formal residual computation are valid. The options-only failure is an API parsing problem.

5.2 How the signature creates the ambiguity

The public definition in the final portion of `FourierCoefficients.wl` has the shape

```
FourierInverseResidual[
  GeneralizedSeries[a_Association],
  cutoff_: Automatic, OptionsPattern[]] := ...
```

The optional positional pattern accepts any expression. A rule such as `"MaxTerms" -> 1000` can therefore occupy the cutoff slot instead of the following options pattern. In the observed match it does, so the internal cutoff checker sees a rule and reports `InvalidCutoff`.^[9]

Wolfram documents `OptionsPattern` as matching sequences or nested lists of immediate and delayed option rules, and documents `Optional` as allowing an argument to be omitted. Their coexistence does not automatically disambiguate an unrestricted positional slot from an option rule.^[25, 26]

This is not evidence that the Fourier coefficient recurrence or its residual normalization is incorrect. The explicit-placeholder call reaches that recurrence and returns a zero residual below the stored relative cutoff. Fixing the entry pattern is both more precise and less disruptive than changing the algebra.

5.3 A narrow implementation change

The tested replacement is

```
FourierInverseResidual[
  GeneralizedSeries[a_Association],
  cutoff : (Automatic | _?exactRealQ) : Automatic,
  OptionsPattern[]] := ...
```

An option rule no longer qualifies as a cutoff, so the optional argument is omitted and `OptionsPattern` receives the rule. Explicit `Automatic` remains admitted. Exact real cutoffs retain the internal positivity check; the pattern does not turn a negative or zero cutoff into a valid one.

This change has a small but real diagnostic tradeoff. A nonnumeric expression explicitly occupying the cutoff position can fall through to the generic invalid-arguments definition instead of reaching the old invalid-cutoff message. That behavior should be chosen deliberately in the public API tests. Two separate overloads, one for options-only calls and one for an explicit cutoff, are an alternative when preserving a more specific malformed-cutoff diagnostic is important.

The candidate does not modify the coefficient computation, cutoff convention, or budget accounting. Additional validation of malformed budget values is outside this small patch and is not counted as another new finding.

5.4 Native candidate results

The signature anchor matched once in the pinned standalone source. The candidate passed the three exercised option forms:

```
FourierInverseResidual[s, "MaxTerms" -> 1000]
FourierInverseResidual[s, {"MaxTerms" -> 1000}]
FourierInverseResidual[s, "MaxTerms" :> (count++; 1000)]
```

Each returned `"ZeroBelowCutoff" -> True`. For the delayed form, a counter initialized to zero ended at one. This rules out accidental repeated evaluation of that particular delayed option in the tested path; it is not a general side-effect theorem for all option expressions.

The immediate operational workaround on the unmodified revision is to supply `Automatic` explicitly before options. The permanent recommendation is the typed signature and focused tests for immediate, delayed, nested, omitted, and explicit cutoff forms on both target runtimes.

6 N3: a true condition loses the exponential-carrier route

6.1 The discrepancy

Create an exact reciprocal source series on the positive side:

```
s = AsymptoticAnalysis`AsymptoticExpansion[
  1/x, {x, 0, 3}, "Backend" -> "Package"];
```

The unconditioned observable succeeds:

```
a = AsymptoticAnalysis`SeriesObservable[s, Exp[z], z];
{Normal[a], a["Remainder"], a["Scale"]}
(* {Exp[1/x], 0, "Factored"} *)
```

The source has $x > 0$ and $z = s(x) = 1/x > 0$. Thus adding $z > 0$ is a valid restriction on exactly this germ. Nevertheless,

```
AsymptoticAnalysis`SeriesObservable[
  s, ConditionalExpression[Exp[z], z > 0], z]
```

returns `Failure["ExponentialScale",...]` on the pinned revision. The message describes an unbounded exponential argument as outside the power-log class and records the reciprocal block $\{-1, 1\}$.

The finite expression is not numerically problematic: $e^{1/x}$ is an exact symbolic carrier and the successful call demonstrates that the package can represent it. The condition does not introduce a new singularity or alter the function on the recorded positive branch.

6.2 Two implementations of exponentiation, selected at the wrong stage

The public observable operation checks specialized forms before it unwraps an outer condition. It compares the original expression e with $\text{Log}[x]$, $\text{Exp}[x]$, and a direct power of x . Only later does it peel conditions into `condition`, prove them on the input jet, and apply the resulting `body`.^[8]

For the plain exponential, the early check selects `seriesExp`. That operation separates non-small power-log blocks into an exact exponential prefactor. For the conditional exponential, the early equality fails, the condition is subsequently proved, and the unwrapped exponential is sent to `seriesJetApply`. Its exponential case uses the generic `fwdExp`, which refuses this unbounded argument.

The call graph therefore contains two different edges for mathematically equivalent admitted observables:

$$\begin{array}{llll} \text{plain } \exp(z) & \longrightarrow & \text{seriesExp} & \longrightarrow \text{exact carrier,} \\ \text{conditioned } \exp(z) & \longrightarrow & \text{seriesJetApply} & \longrightarrow \text{fwdExp} \longrightarrow \text{refusal.} \end{array}$$

This explains the failure without assuming anything about Wolfram’s native asymptotic coverage. It also distinguishes N3 from earlier conditional-input backend issues.

6.3 The required semantic invariant

Let S denote the input germ, let H be an observable, and let C be a condition proved on that germ. A useful dispatch invariant is

$$\text{Obs}(S, \text{ConditionalExpression}(H, C)) \equiv \text{Obs}(S, H) \quad \text{on the admitted germ,} \quad (2)$$

where the conditioned result may retain additional provenance and restrictions. The invariant does *not* say that arbitrary conditions may be deleted. It applies only after the existing proof step succeeds.

For the witness, the proof is elementary: $x > 0$ implies $1/x > 0$. The exact observable is $e^{1/x}$ and its internal factored bracket is the constant 1. There is no Taylor remainder to invent or strengthen. The failure is therefore a missed representation route, not insufficient input precision.

6.4 Candidate change after condition proof

The candidate keeps the existing fast paths for unconditioned expressions. After conditions have been extracted and proved, it adds specialized dispatch on `body`:

```
result = Which[
  body === Log[x], seriesLog[s, OptionValue["Cutoff"], limit],
  body === Exp[x], seriesExp[s, OptionValue["Cutoff"], limit],
  Head[body] === Power && body[[1]] === x && FreeQ[body[[2]], x],
    seriesPower[s, body[[2]], OptionValue["Cutoff"], limit],
  True,
    j = seriesJetApply[body, x, d["Jet"], d, h, limit];
    seriesMake[Join[d, <|"Jet" -> j|>],
      {"Observable", {s}, e, x}, h];
```

The requested cutoff is passed to the specialized operation, rather than replacing `Automatic` with a prematurely resolved working cutoff. This preserves the specialized operation’s own cutoff policy.

The result’s recipe is then explicitly set to

```
{"Observable", {s}, e, x}
```

using the original conditioned expression e . Otherwise the specialized result could retain only an `Exp` recipe and lose the restriction on later replay. The ordinary public decoration of inverse-function branch selections and provenance remains in place.

The delivered change does not remove the condition checker, apply unrestricted logarithmic identities, or replace an unknown truth value by `True`. It reuses a representation route that the same operation already exposes for the plain observable.

6.5 Executed results and scope limits

On Wolfram 15, the patched conditional example returned $e^{1/x}$ with zero remainder. Its stored recipe retained the original `ConditionalExpression[Exp[z], z > 0]`. The negative control

```
SeriesObservable[s, ConditionalExpression[Exp[z], z < 0], z]
```

still returned `Failure["IncompatibleObservableCondition",...]`.

These checks establish the reproduced case and a necessary safety property. They do not establish all conditional observables. In particular, the current public path still flattens the input representation before reaching the new post-proof dispatch. A nonflattenable factored input can therefore require a more general, representation-aware condition proof before a conditional specialized logarithm or power becomes available. The minimal candidate is deliberately not advertised as that complete redesign.

A successful runtime result for refinement of the patched conditional expression was not obtained in this review. Recipe retention was directly inspected; actual refinement replay remains a proposed acceptance test. This is why the delivered evidence does not report an end-to-end refinement pass.

7 Numerical and structural controls

7.1 The 42-case screen

The executed screen used six real positive-side germs:

$$\begin{aligned} g_1(x) &= x + x^2 \log x, & g_2(x) &= x^2 + x^3 \log x, \\ g_3(x) &= x + x^{3/2}, & g_4(x) &= x^{1/2} + x \log x, \\ g_5(x) &= x + x^2 (\log x)^2, & g_6(x) &= x - x^3. \end{aligned}$$

For each g_j , it requested cutoff 3 from the package backend for

$$e^{g_j}, \quad \sin g_j, \quad \cos g_j, \quad \log(1 + g_j), \quad (1 + g_j)^{1/3}, \quad (1 + g_j)^{-3}, \quad g_j^2.$$

These cases exercise fractional weights, logarithmic coefficients, cancellation, analytic composition, and positive and negative powers without entering a separate unsupported scale family.

For a nonexact result with finite part $A(x)$ and recorded remainder scale $B(x)$, the diagnostic evaluated

$$Q_k = \frac{F(10^{-k}) - A(10^{-k})}{B(10^{-k})}, \quad k \in \{6, 12, 24\}, \quad (3)$$

with 100-digit numerical evaluation. It flagged a case when all three values were numeric and

$$|Q_{24}| > 1000 \max(1, |Q_6|).$$

For an exact result, it instead attempted to simplify $F - A$ under $x > 0$.

The returned summary was

```
{42, <|"BoundedCheck" -> 41, "Exact" -> 1|>, {}, {}}
```

The empty lists denote no nonzero exact-check result under the implemented test and no flagged numerical growth ratio. Every case returned a series object within the submitted per-case budget.

7.2 What the screen establishes, and what it does not

The result is useful because it checks ordinary arithmetic independently of the API failures. N1 does not become a claim that every forward expansion is wrong; N2 does not become a claim that Fourier coefficients fail; N3 does not become a claim that exact exponential carriers are unavailable.

Nevertheless, sampling three points cannot prove $F - A = O(B)$, and a permissive growth threshold can miss slow or intermittent violations. Finite precision can also hide sufficiently small differences. A returned numerical value is not an outward enclosure, and no certificate flag is attached to this screen. The report therefore calls the 41 cases *nonexact diagnostic checks*, not 41 proved remainder bounds.

The supplied runner retains the same sample points and threshold, making the scope inspectable. It prints all observations rather than reducing every result to an unqualified pass label. A different machine can also time out under the same wall-clock limits; such a timeout must be recorded as a timeout, not as a coefficient failure or success.

7.3 Patch checks are separate from arithmetic controls

The targeted candidate observations were obtained in temporary generated-package copies. N2 and N3 were exercised together in one returned batch, with a source-variable guard also present. The final N1 source and ordinary-target guards were exercised in a separate returned batch. A later combined batch did not return a usable result and is not counted.

The archive's Python staging code has six passing local unit tests on synthetic source fragments. Those tests cover unique anchors, refusal of missing or repeated anchors, nonmutation of input files, refusal of an output directory inside the checkout, and avoiding output publication when input preparation fails. They do not test Wolfram syntax, Mathics evaluation, or the full repository build.

This separation prevents a common evidence error: treating a passing patch utility, an independent arithmetic screen, and a partially completed kernel batch as one comprehensive integration test.

8 Wolfram 15 and Mathics3: conclusions justified here

8.1 Wolfram 15

The key findings are native Wolfram 15 observations, not simulations in Python or a substituted evaluator. The service loaded the pinned generated package and returned the reproductions and the focused candidate outputs described above.

The relevant comparison with built-in Wolfram behavior is narrow and direct. Native **Series** rejects the **Pi** variable; the package's own branch returns an object. Native option-pattern rules explain why the Fourier signature needs a constraint. The successful plain observable demonstrates that the package

already has a capability the conditional route misses. No general feature-ranking table is needed to establish these points.[24, 25, 26]

Wolfram 15 execution does not establish the entire package’s supported API surface. The full repository test suite, all native-backend branches, installer workflows, and documentation examples were not run. The final artifact is an incremental audit, not a release certification.

8.2 Mathics3

The package does not simply delegate all behavior to the host evaluator. The inspected compatibility layer introduces bounded helpers in a separate context, adapts selected operations, and creates names for otherwise unavailable system-level forms. The source also has Mathics-specific simplification, calculus, Taylor, special-function, and certificate adjustments.[18, 19, 20, 21, 22]

This architecture creates three specific obligations for the new fixes. First, the N1 variable policy must recognize numeric constants without depending on the accidental availability of every numeric evaluator. That motivates qualified inert-constant checks, but the candidate list is only a starting point. Second, the N2 signature must be tested in Mathics’ actual matcher with nested and delayed options, not only with Wolfram’s matcher. Third, the N3 condition proof and specialized exponential operation must both succeed under the Mathics adapters; the truth of $1/x > 0$ on $x > 0$ does not alone prove that its symbolic implementation reaches the same code path.

No Mathics version string or Mathics package-test output was obtained. Accordingly, the current evidence supports “**Mathics source-reviewed; runtime acceptance pending**”, not either “works on Mathics3” or “fails on Mathics3.” The runnable test file is designed to avoid a dependency on a particular test-report framework, but even that intended portability remains to be executed.

8.3 A focused acceptance matrix

Acceptance question	Wolfram 15	Mathics3
Pinned package loads	Observed	Not executed
Baseline N1, N2, N3 witnesses reproduce	Observed	Not executed
N1 source and ordinary inverse-target guards	Observed separately	Not executed
N1 user-context name control	Observed	Not executed
N2 immediate, nested, delayed options	Observed with candidate	Not executed
N3 true and false conditions	Observed with candidate	Not executed
N3 original conditioned recipe retained	Observed with candidate	Not executed
N3 refinement replay	Pending	Pending
All four source edits in one completed integration batch	Not established	Not established
Full repository tests and rebuilt modular/standalone parity	Not executed	Not executed

The unfinished rows are explicit deliverable boundaries. They are not new findings about the repository’s existing continuous-integration process.

9 Fix integration and focused development priorities

9.1 A staged source patch rather than an opaque replacement package

The supplied `code/candidate_patch.py` reads the three canonical files, verifies that each of four source anchors occurs exactly once, and writes revised fragments plus a unified diff into a new directory outside the input checkout. It verifies the expected Git revision by default. An explicit override supports a source archive lacking Git metadata, but the exact anchors must still match.

The tool stages only the changed files, not a runnable full package. This prevents an incomplete directory from being mistaken for a new distribution. It makes no network requests, does not update the user's repository, and refuses to overwrite an existing output directory.

The appropriate integration workflow is to inspect the diff, apply it to a disposable checkout of the pinned revision, regenerate the standalone artifact using the repository's own builder, and run the existing validation gates together with the new narrow tests. The current builder assembles the canonical source and preserves the streaming package-context behavior; editing only the generated file would bypass that source of truth.^[23]

The runtime candidate tests used temporary standalone copies to test behavior quickly. They did not themselves execute the canonical rebuild workflow. The distinction is recorded rather than hidden behind a claim that a patched package has been released.

9.2 Recommended order of work

First, fix the N2 signature. It is the smallest isolated change, has an immediately usable workaround, and has direct ordinary, nested, and delayed-option candidate results. Preserve the stored cutoff convention and decide how malformed explicit cutoff expressions should be diagnosed.

Second, fix N3's post-proof dispatch. Retain the condition proof and the original observable recipe. Require a false-condition control in the same test group as the true-condition success. Before treating this as complete conditional-observable support, add refinement replay and investigate nonflattenable input representations separately.

Third, finish the N1 admission policy at all entry points. The common-chart and ordinary-inverse edits fix the demonstrated examples, but this is the change with the widest public-API implications. Enumerate source variables, inverse targets, aliases, and specialized constructors; test true numeric constants and valid look-alike user symbols in each. The acceptance criterion is consistent rejection or admission according to the documented policy, not merely absence of a downstream message.

These are concrete extensions of the new findings. They do not replace the repository's already recorded development roadmap with another generic request for more tests, more abstractions, or more special functions.

9.3 Performance implications without invented speedups

No new benchmark-backed performance defect is asserted in this report. Existing performance topics encountered during inspection were excluded rather than relabeled. The single observed load duration is not used to compare runtimes or promise an optimization.

For the proposed changes, the main performance consideration is avoiding unnecessary work on malformed or misrouted requests. N1 rejects a numeric variable before building coefficient data. N2 lets a rule reach option resolution rather than a doomed cutoff check. N3 reuses the exact-carrier operation instead of entering a generic route that is known to fail for the witness.

These are control-flow improvements, not measured speedups. The cost of a semantic predicate such as

`NumericQ` or `exactRealQ` can depend on the expression and evaluator, so an unconditional constant-time complexity claim would be unjustified. If profiling shows predicate overhead at a hot entry point, it should be evaluated on valid workloads as well as rejected ones; the current evidence does not require such an optimization.

9.4 Precise completion criteria

A completed N1 implementation must refuse numeric source and target symbols on every admitted route while preserving ordinary nonnumeric symbols, including names that resemble constants or options. A completed N2 implementation must give the same residual result for options-only and explicit-`Automatic` forms, with delayed option evaluation behavior checked. A completed N3 implementation must preserve the specialized observable capability after a condition is proved, retain that condition for replay, and continue to refuse false or unproved restrictions.

Those statements are falsifiable and local to the findings. They are preferable to declaring the fixes complete because one sample notebook no longer displays a warning.

10 Using the archive

The package contains the article source and PDF, runnable Wolfram Language observations and controls, a Python source-patch stager, its unit tests, an offline novelty scanner, and evidence records. The evidence records are curated transcriptions and summaries of returned tool outputs; they are not represented as unedited complete kernel session logs.

A local reproduction session can use:

```
$AuditPackage = "/absolute/path/to/AsymptoticAnalysis.wl";
Get["/absolute/path/to/code/reproduce.wl"];
```

Use a fresh kernel for the unmodified and modified package. A command-line environment can set `ASYMPTOTIC_PACKAGE` instead. The script prints observations and stores them in `$AuditResults`; an optional string `$AuditOutput` requests a text export. The runner does not silently download or replace the package.

The numerical screen is a separate invocation of `code/run_controls.wl`. Do not interpret its output as a rigorous bound certificate. The source-patch stager is invoked with Python 3.10 or newer:

```
python code/candidate_patch.py /path/to/checkout /path/to/new-staging-dir
python -m unittest discover -s code -p 'test_candidate_patch.py' -v
python code/novelty_scan.py /path/to/checkout --output additional-screen.json
```

The novelty scanner reads local `.tex` and `.md` files only and preserves potential matches for manual inspection. Its absence of a match is not a semantic novelty certificate.

11 Conclusion

The pinned repository has three newly reproduced public-boundary defects that warrant focused changes. The strongest actionable result is the before/after relationship: a malformed numeric coordinate should be refused rather than packaged as a germ; a documented option should not become a cutoff; and a proved condition should not remove an already available exact-carrier observable route.

The native Wolfram 15 observations, candidate checks, and 42-case diagnostic screen support those conclusions within their stated scope. Mathics3 remains an unexecuted runtime target in this review, and the complete combined rebuild and validation remain integration work. The supplied tests and source patches make those remaining checks concrete without presenting them as completed.

A Source coverage and claim boundaries

The following is an inspection map, not a count of files automatically certified correct. “Targeted ranges” means the relevant definitions and adjacent consumers were read; it does not mean every line of that module was reviewed.

File or group under <code>src/Kernel</code>	Review purpose and boundary
<code>AsymptoticAnalysis.wl</code>	Targeted ranges: exactness and coefficient helpers, coordinate construction, public dispatch, ordinary inverse variable checks. N1 is anchored here.
<code>SeriesOperations.wl</code>	Representation, arithmetic, observables, composition scope, differentiation and refinement definitions. N3 is anchored here.
<code>FourierCoefficients.wl</code>	Finite mode algebra, coefficient construction, source normalization, residual and coefficient APIs. N2 is anchored here.
<code>LambertInverse.wl</code> ; <code>CorePerturbation.wl</code> ; <code>FlatSectors.wl</code>	Branch and observable conventions, exact-core versus finite-sector data, remainder metadata; no additional new defect asserted.
<code>GammaInverse.wl</code> ; <code>BarnesForward.wl</code>	Targeted ordered inverse data and finite logarithmic forward model; no full runtime family sweep.
<code>DirichletSpecialFunctions.wl</code> ; <code>ParameterizedSpecialFunctions.wl</code> ; <code>SpecialFunctionRealDomain.wl</code> ; <code>SpecialFunctionAdapters.wl</code> ; <code>InverseCertificates.wl</code>	Fixed-parameter assumptions, admissible arguments, finite Taylor/Dirichlet construction, realness boundaries. Adapter transformations and numerical checks; targeted certificate arithmetic and source-domain handling. No new certificate theorem claimed.
<code>MathicsCompatibility.wl</code> ; <code>MathicsAssumptions.wl</code> ; <code>MathicsSimplification.wl</code>	Interpreter-specific symbols, assumptions, and simplification paths; source review only.
<code>MathicsAlgebra.wl</code> ; <code>MathicsTaylor.wl</code> ; <code>MathicsCalculus.wl</code>	Bounded supporting operations and defining Taylor forms; source review only.
<code>MathicsCoreFunctions.wl</code> ; <code>MathicsSpecialFunctions.wl</code> ; <code>MathicsCertificate.wl</code> ; <code>MathicsCalls.wl</code>	Selected late transformations and helper behavior; source review only.
<code>validation/build_standalone.py</code>	Canonical-source assembly and streaming Mathics bootstrap structure; no full local repository rebuild.

The absence of a source group from this table should not be read as a pass. In particular, the report

does not claim exhaustive validation of all release scripts, CI jobs, documentation builds, serialized object shapes, or combinations of specialized scale families.

B Condensed returned evidence

The evidence folder includes machine-readable status records. The core returned facts are summarized here for convenient reading.

Batch	Returned observation
Baseline N1	Forward Pi : $1-\text{Pi}+\text{Pi}^2$, variable Pi , nonzero formal remainder; ordinary inverse target Pi : $\text{Pi}-\text{Pi}^2$. Invalid native <code>SeriesData</code> variable messages occurred.
Baseline N2	Options-only <code>MaxTerms->1000</code> : <code>InvalidCutoff</code> . Explicit <code>Automatic</code> : zero residual below relative cutoff 2.
Baseline N3	Plain exponential: exact factored $\text{Exp}[1/x]$. True conditioned exponential: <code>ExponentialScale</code> .
Candidate N2/N3	Immediate and nested options returned zero residual; delayed option evaluated once. Conditioned exponential returned exact $\text{Exp}[1/x]$; original conditioned recipe retained; false condition returned <code>IncompatibleObservableCondition</code> .
Final candidate N1	Both anchors matched once; source Pi and ordinary target Pi returned structured failures. User-context Pi and the sine control returned expected finite parts.
Arithmetic screen	42 returned series objects: 41 nonexact diagnostic cases and one exact case; no flags under the stated test.
Python staging tests	Six local synthetic-fragment unit tests passed. Not package-runtime tests.

A combined candidate batch and a refinement candidate request did not yield a completed service result. Their absence is not converted into either success or a repository failure. Mathics package execution was not obtained.

C Proposed additional acceptance cases

The following cases are included as an integration specification, not as an executed matrix. For N1, extend the numeric-coordinate test to `E`, `EulerGamma`, supported inert constants, assigned symbols, public aliases, and specialized inverse targets; retain look-alike user-context symbols as controls. For N2, test omitted, explicit, zero, negative, malformed, and exact algebraic cutoffs together with immediate, delayed, and nested options. For N3, test false and unresolved parameter conditions, explicit cutoffs, nonflattenable source representations, and refinement replay of the original conditioned recipe.

Run these on the actual Wolfram and Mathics versions intended for release. For every case, record the runtime version, whether an object or a failure was returned, relevant finite-part and remainder fields, and unexpected messages. A skipped or timed-out case must remain visibly distinct from an accepted result.

References

- [1] Asymptotic contributors. [Repository README](#), reviewed commit `8cee870994f5`.
- [2] Asymptotic contributors. [Kernel module guide](#), same revision.
- [3] Asymptotic contributors. [Code-review status and implementation register](#), same revision.
- [4] Asymptotic contributors. [Wave-three intake](#), same revision.
- [5] Asymptotic contributors. [Wave-four intake](#), same revision.
- [6] Asymptotic contributors. [Wave-five review index](#), same revision.
- [7] Asymptotic contributors. [src/Kernel/AsymptoticAnalysis.wl](#). In particular, `exactness` helpers, `localCoordinate`, and the ordinary inverse check at line 1011.
- [8] Asymptotic contributors. [src/Kernel/SeriesOperations.wl](#). In particular, `seriesData`, `seriesMake`, `seriesExp`, `seriesJetApply`, and `SeriesObservable`.
- [9] Asymptotic contributors. [src/Kernel/FourierCoefficients.wl](#). In particular, the public residual definition and its unrestricted optional cutoff in the reviewed source.
- [10] Asymptotic contributors. [CorePerturbation.wl](#).
- [11] Asymptotic contributors. [FlatSectors.wl](#).
- [12] Asymptotic contributors. [LambertInverse.wl](#).
- [13] Asymptotic contributors. [GammaInverse.wl](#); [BarnesForward.wl](#).
- [14] Asymptotic contributors. [DirichletSpecialFunctions.wl](#).
- [15] Asymptotic contributors. [SpecialFunctionAdapters.wl](#); [InverseCertificates.wl](#).
- [16] Asymptotic contributors. [SpecialFunctionRealDomain.wl](#).
- [17] Asymptotic contributors. [ParameterizedSpecialFunctions.wl](#).
- [18] Asymptotic contributors. [MathicsCompatibility.wl](#); [MathicsAssumptions.wl](#).
- [19] Asymptotic contributors. [MathicsSimplification.wl](#).
- [20] Asymptotic contributors. [MathicsTaylor.wl](#); [MathicsAlgebra.wl](#).
- [21] Asymptotic contributors. [MathicsCalculus.wl](#); [MathicsCoreFunctions.wl](#).
- [22] Asymptotic contributors. [MathicsSpecialFunctions.wl](#); [MathicsCertificate.wl](#); [MathicsCalls.wl](#).
- [23] Asymptotic contributors. [validation/build_standalone.py](#).
- [24] Wolfram Research. *Series*. Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/Series.html>. Accessed September 10, 2026.
- [25] Wolfram Research. *OptionsPattern*. Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/OptionsPattern.html>. Accessed September 10, 2026.
- [26] Wolfram Research. *Optional*. Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/Optional.html>. Accessed September 10, 2026.